Pontifícia Universidade Católica de Minas Gerais

Fabio Walder, Ian Campos, João Paulo Gomes e Márcio Gazzinelli

Sprint 03 - Armazenamento / Persistência

Chefim - Seu Companheiro na Cozinha

Belo Horizonte
2026

1. Visão geral

A Sprint 3 do Chefim teve como objetivo implementar a persistência de dados e integrar o banco de dados às funcionalidades do aplicativo, por meio das operações CRUD (criação, leitura, atualização e exclusão).

Para isso, foram utilizadas duas tecnologias principais: SQLite, responsável pelo armazenamento dos dados estruturados, e SharedPreferences, utilizado para salvar configurações e informações da sessão do usuário.

Com essa implementação, o aplicativo passou a preservar os cadastros e as alterações realizadas, mesmo depois de ser fechado e aberto novamente.

2. Projeto do banco de dados – SQLite

O SQLite foi utilizado como banco de dados local do Chefim, permitindo armazenar informações diretamente no dispositivo, sem necessidade de conexão com a internet.

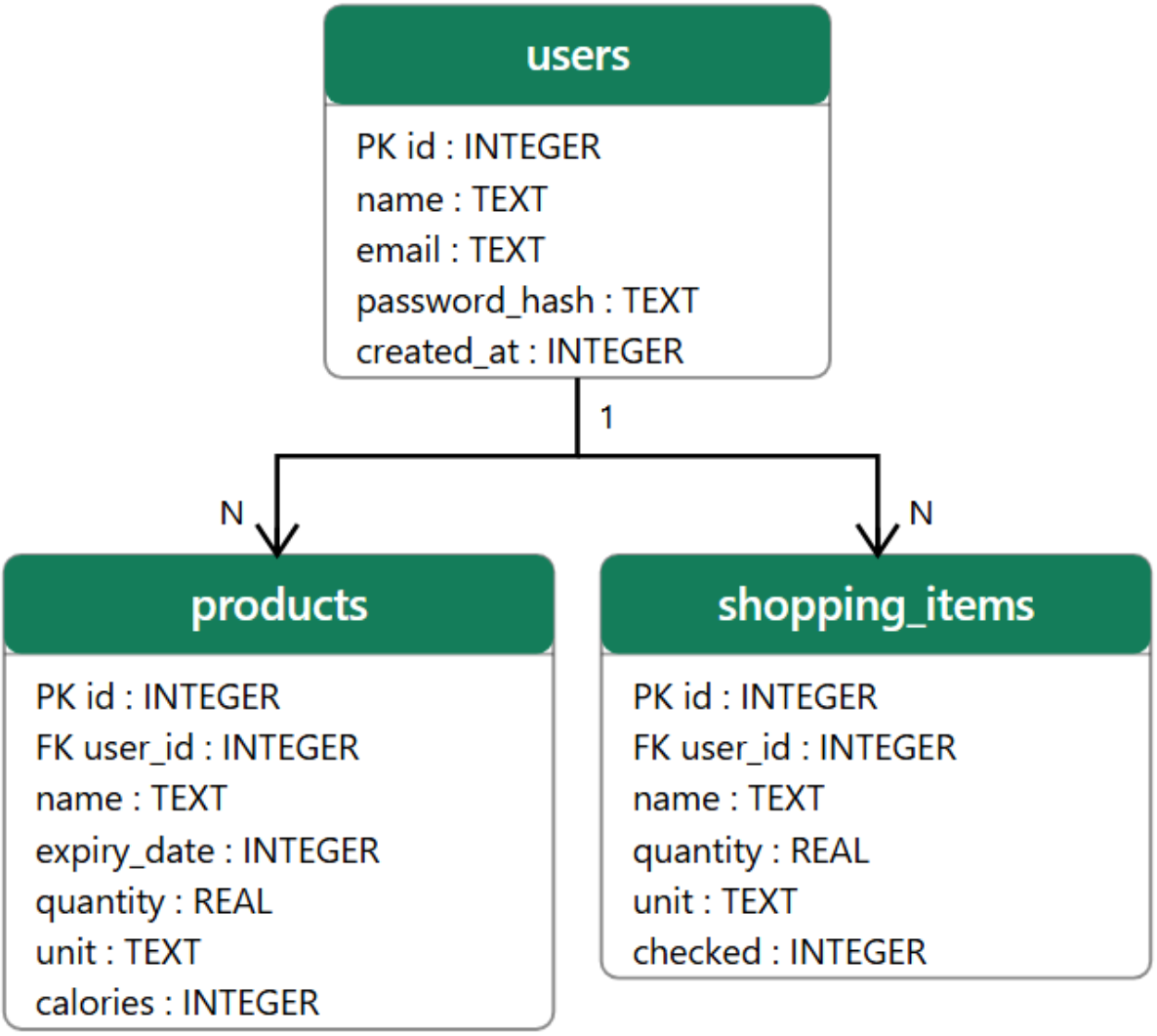
A integração com Flutter foi realizada utilizando a biblioteca sqflite. O banco, denominado chefim.db, foi estruturado em três tabelas principais, cada uma relacionada a uma funcionalidade do aplicativo.

Tabela	Funcionalidade	Dados armazenados
users	Cadastro e login	ID, nome, e-mail, hash da senha e data de cadastro
products	Despensa	ID, usuário, nome, validade, quantidade, unidade e calorias
shopping_items	Lista de compras	ID, usuário, nome, quantidade, unidade e situação de compra

A tabela users armazena as informações necessárias para identificar e autenticar os usuários. As tabelas products e shopping_items possuem uma chave estrangeira (user_id) relacionada ao identificador da tabela users, permitindo que cada conta tenha sua própria despensa e lista de compras.

Esses relacionamentos são do tipo um para muitos (1:N), pois um usuário pode possuir vários produtos e vários itens de compras. A estrutura completa é apresentada no diagrama entidade-relacionamento (DER).

2.1. Diagrama entidade-relacionamento (DER)



O gerenciamento do banco é centralizado no arquivo database_service.dart, responsável pela criação das tabelas, gerenciamento da estrutura e execução das operações CRUD.

3. Integração do SQLite com o aplicativo

3.1. Cadastro e login – Tabela users

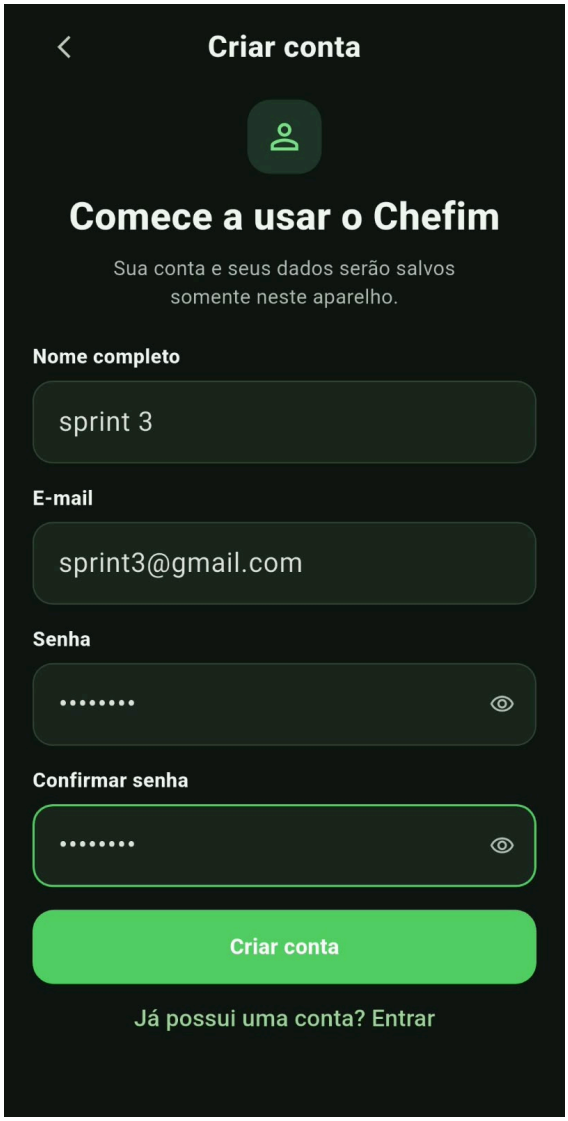
A tabela users foi integrada às telas de cadastro e login, permitindo que as contas sejam armazenadas localmente no SQLite.

No cadastro, o usuário informa nome, e-mail e senha. O aplicativo verifica a validade das informações, exige uma senha de pelo menos oito caracteres e impede o cadastro de e-mails duplicados.

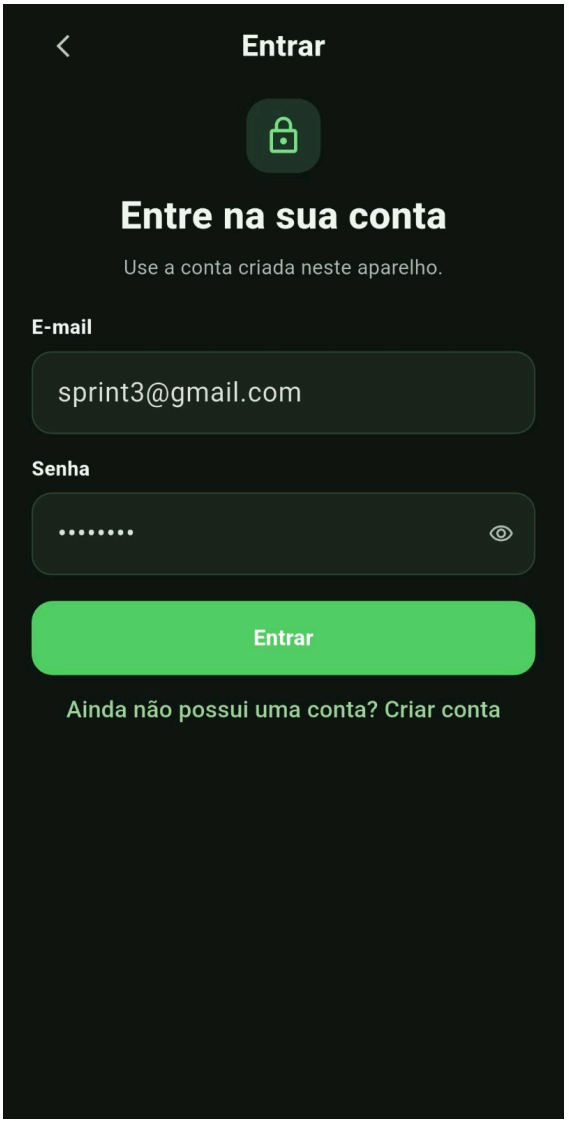
As senhas são protegidas utilizando hash BCrypt, evitando seu armazenamento em texto simples.

No login, o aplicativo consulta o e-mail cadastrado na tabela users e verifica a senha informada. Após a autenticação, a sessão é mantida utilizando SharedPreferences, permitindo que o usuário continue conectado mesmo após fechar o aplicativo.

Essa solução de autenticação local é temporária, enquanto o projeto ainda não utiliza o Supabase.



Tela de cadastro de usuário



Tela de login

3.2. Despensa – Tabela products

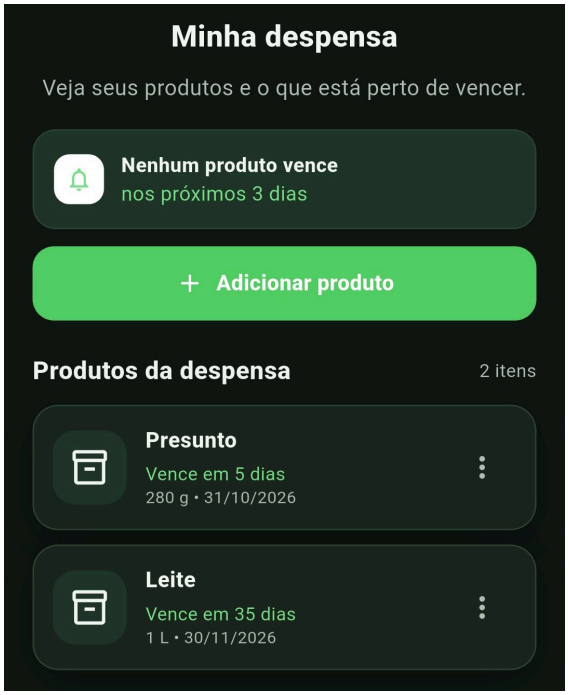
A tabela `products` foi integrada à tela de despensa, permitindo realizar todas as operações CRUD:

- **Create:** cadastrar produtos com nome, quantidade, unidade e data de validade.
- **Read:** consultar os produtos cadastrados e acompanhar seus vencimentos.
- **Update:** editar as informações de produtos existentes.
- **Delete:** excluir produtos da despensa.

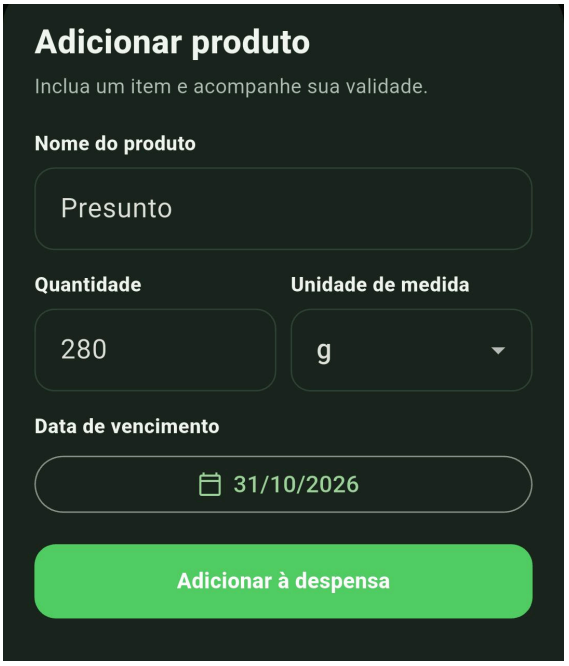
Os registros são associados ao usuário por meio do campo user_id, permitindo que cada conta tenha sua própria despensa.

Além disso, os dados cadastrados são utilizados para atualizar o resumo da tela inicial, incluindo a quantidade de produtos e os avisos de validade.

Todas as alterações são armazenadas no SQLite, permitindo recuperar os produtos mesmo após fechar e abrir novamente o aplicativo.



Tela de despensa com a lista de produtos



Cadastro e edição de produto

3.3. Lista de compras – Tabela shopping_items

A tabela `shopping_items` foi integrada à lista de compras, possibilitando armazenar e gerenciar os itens que o usuário deseja comprar.

Foram implementadas as seguintes operações CRUD:

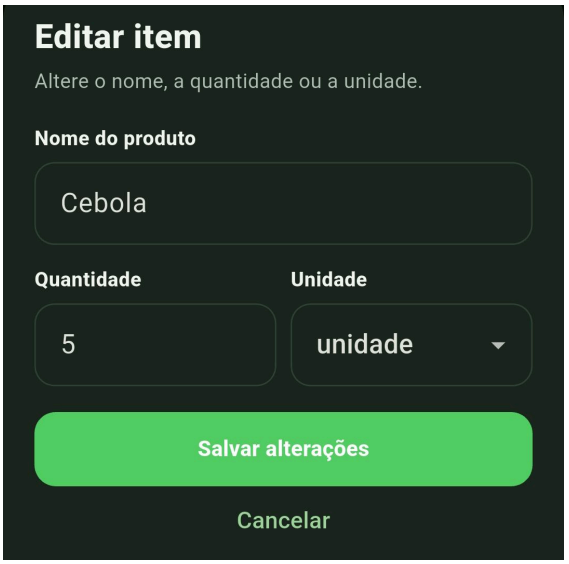
- **Create:** adicionar itens com nome, quantidade e unidade.
- **Read:** consultar os itens armazenados.
- **Update:** editar as informações e marcar ou desmarcar itens como comprados.
- **Delete:** excluir itens individualmente ou limpar toda a lista.

O campo checked indica se o item foi marcado como comprado, enquanto o campo user_id associa os registros à conta correspondente.

Assim como na despensa, todas as alterações permanecem armazenadas no SQLite, mesmo após o encerramento do aplicativo.



Tela da lista de compras



Edição de item

4. SharedPreferences – Configurações e sessão

O SharedPreferences foi utilizado para armazenar configurações simples do usuário, permitindo que suas escolhas permaneçam salvas mesmo após o aplicativo ser fechado.

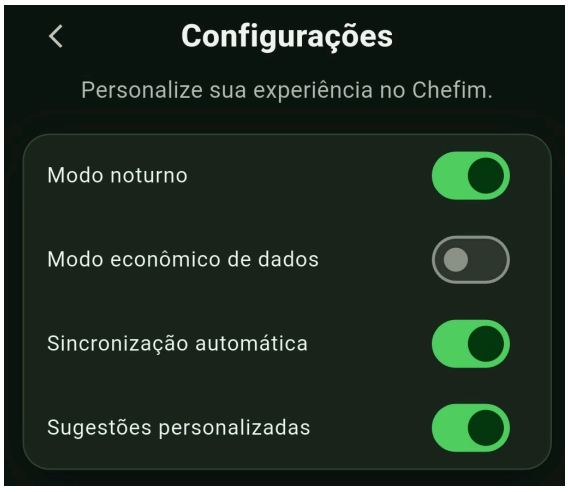
As preferências armazenadas incluem:

- Tema claro ou escuro;
- Ativação dos alertas de validade;
- Quantidade de dias de antecedência dos avisos;
- Avisos de novas receitas;
- Lembretes da lista de compras;
- Modo econômico de dados;
- Sincronização automática;
- Sugestões personalizadas.

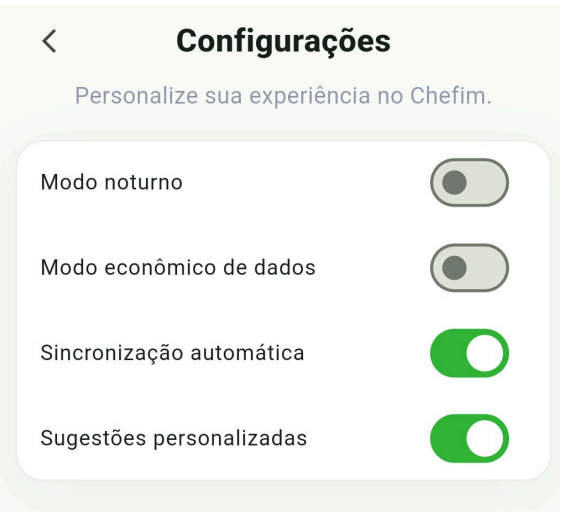
O tema claro/escuro já altera efetivamente a aparência do aplicativo. As demais preferências são armazenadas, mas os mecanismos reais de notificações, sincronização e sugestões personalizadas ainda não foram desenvolvidos.

O SharedPreferences também é utilizado para salvar o identificador do usuário autenticado. Dessa forma, ao abrir o aplicativo novamente, a conta é recuperada a partir dos dados armazenados no SQLite, mantendo o usuário conectado.

Ao realizar logout, o identificador da sessão é removido, mas os dados da conta continuam preservados no banco.



Configurações do aplicativo com o tema escuro ativado



Configurações do aplicativo com o tema escuro desativado